

Obiettivo

Il gruppo si propone di realizzare **BioAssault**, un clone semplificato del videogioco *Vampire Survivors*, mantenendone le meccaniche principali ma ambientandolo in un contesto biologico. In particolare, il giocatore controllerà una cellula che dovrà sopravvivere il più a lungo possibile all'interno di un'arena 2D, contrastando ondate progressive di virus che aumentano in numero e pericolosità con il passare del tempo.

L'obiettivo del progetto è costruire una struttura di gioco completa nelle sue funzionalità essenziali: movimento del personaggio, presenza di nemici differenziati, sistema di combattimento automatico, gestione dell'esperienza e aumento progressivo della difficoltà. A tali elementi si affiancano alcune funzionalità opzionali, pensate per arricchire l'esperienza di gioco, come nemici potenziati, musica di sottofondo e statistiche finali dettagliate.

1.1 Descrizione e requisiti

Requisiti funzionali

Le funzionalità minime ritenute obbligatorie sono le seguenti:

- Presenza di un menù di avvio per l'accesso alla partitaImplementazione di un'arena 2D con limiti definiti e bordi invalicabili, comprensiva della gestione della fine della partita
- Implementazione del personaggio principale con possibilità di movimento libero all'interno dell'arena
- Implementazione di almeno due tipologie di nemici, caratterizzate da comportamenti differenti.
- Implementazione di un sistema di collisioni tra personaggio, nemici e proiettili.
- Implementazione di almeno due armi differenti.
- Implementazione del sistema di danno e della gestione delle vite
- Implementazione del sistema di esperienza con avanzamento di livello e scelta tra tre possibili upgrade.
- Visualizzazione in sovraimpressione delle principali informazioni di gioco, tra cui vite, livello, esperienza e tempo di sopravvivenza.
- Implementazione di un sistema di spawn progressivo dei nemici, con aumento della difficoltà nel tempo.

Le funzionalità opzionali previste sono:

- Implementazione di un nemico di tipo élite o boss.
- Implementazione di abilità passive aggiuntive.
- Implementazione di statistiche finali dettagliate al termine della partita
- Implementazione di musica di sottofondo ed effetti sonori.
- Implementazione di un secondo personaggio giocabile con statistiche differenti.

Requisiti non funzionali e principali sfide

Oltre alle funzionalità di gioco, il progetto richiede una corretta organizzazione del lavoro di gruppo e un uso adeguato degli strumenti di sviluppo collaborativo. In particolare, particolare attenzione dovrà essere posta alla cooperazione tra i membri del gruppo, all'utilizzo di Git per la gestione del codice sorgente e all'adozione coerente di una struttura architettuale ispirata al pattern MVC, così da mantenere separata la logica di gioco dalla componente grafica.

Ulteriori aspetti rilevanti riguardano la corretta gestione del ciclo di aggiornamento del gioco, la coerenza del sistema di collisioni e combattimento e la capacità di mantenere il progetto estendibile e comprensibile anche in presenza di più sottocomponenti sviluppate in parallelo

Suddivisione del lavoro

La suddivisione iniziale del lavoro all'interno del gruppo è la seguente:

- **Valentina:** implementazione dell'engine di gioco, gestione del ciclo di aggiornamento, sistema di spawn e incremento progressivo della difficoltà
- **Francesco:** implementazione del sistema di combattimento base, comprensivo di HP, danno, collisioni, modellazione di almeno un'arma e dei relativi proiettili.
- **Jonathan:** gestione degli input e implementazione della parte grafica dell'arena, dell'HUD e delle schermate di gioco.
- **Rebecca:** implementazione del sistema di esperienza e level-up, gestione degli upgrade e delle statistiche finali.

1.2 Modello del Dominio

Il giocatore verrà posizionato su un'arena ben delineata e non generata in modo casuale e dovrà essere in grado di potersi muovere nelle 4 dimensioni di un mondo 2D.

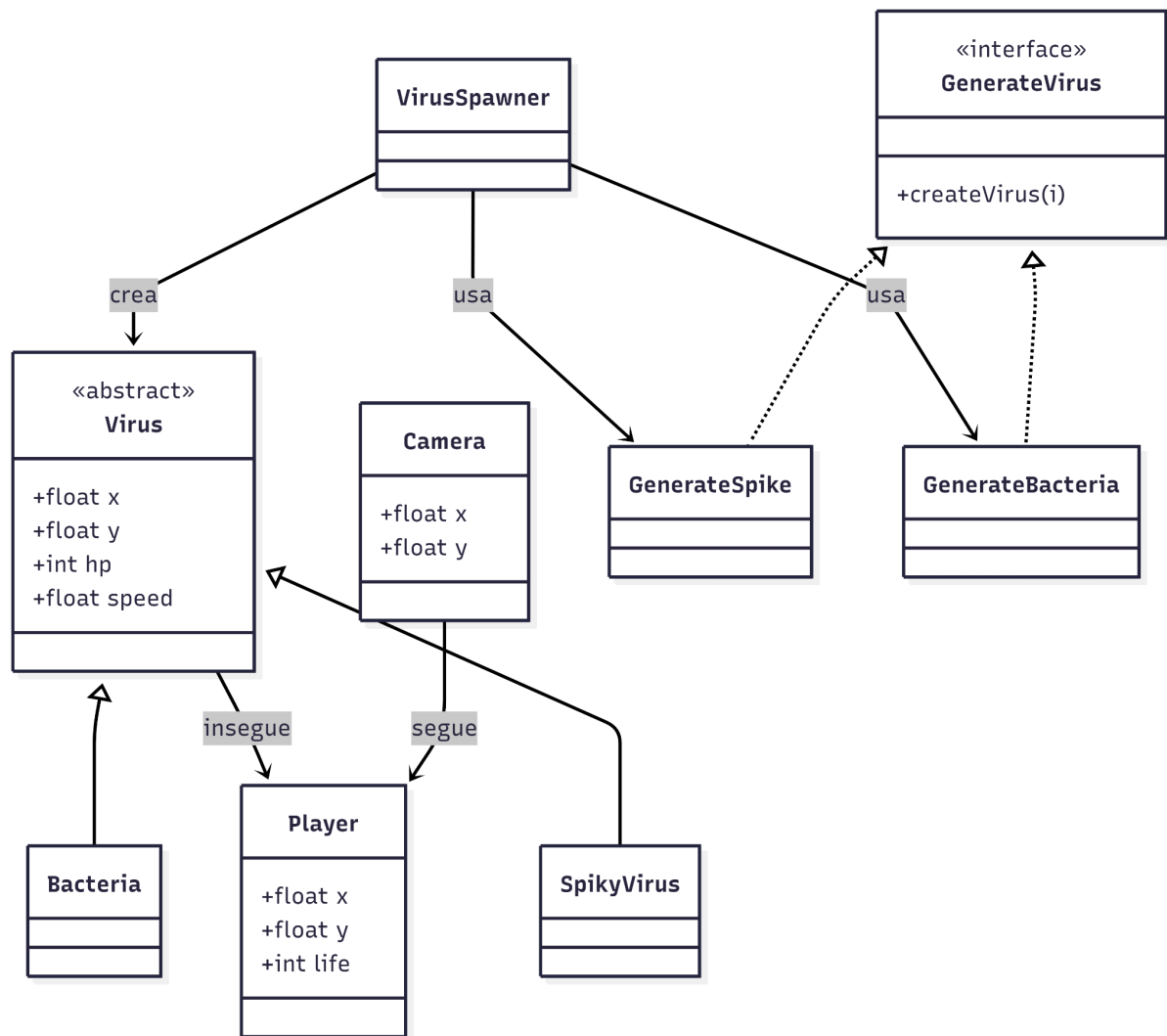
L'arena di gioco dovrà avere dei bordi per impedire l'uscita del giocatore.

Attorno al giocatore compariranno dei nemici, qui identificati come virus, che tenteranno di raggiungerlo per poterlo attaccare.

I nemici dovranno essere in grado di trovare il giocatore e seguirlo nei suoi spostamenti.

Per difendersi il player sarà in grado di sparare dei proiettili. I proiettili avranno l'abilità di seguire il virus nemico e causare danno.

Sopra ad ogni nemico è prevista una barra della vita che diminuirà in seguito ai colpi di proiettile.



2 Design

2.1 Architettura

Il progetto adotta un'architettura ispirata al pattern MVC, adattata al contesto di un videogioco real-time tramite un game loop centrale.

L'organizzazione del codice presenta una netta separazione tra logica di gioco, visualizzazione e gestione dell'interazione, mentre il coordinamento generale avviene attraverso un game loop centrale.

Model contiene lo stato e il comportamento delle entità di gioco.

Rientrano le classi come *Handler* che mantiene la collezione degli oggetti presenti nella partita ed esegue i metodi `tick()` e `render()` su ciascuno di essi, le classi del dominio come `Player`, `Virus` e `VirusSpawner` implementano, rispettivamente, il comportamento del personaggio, del nemico e della loro generazione nel tempo.

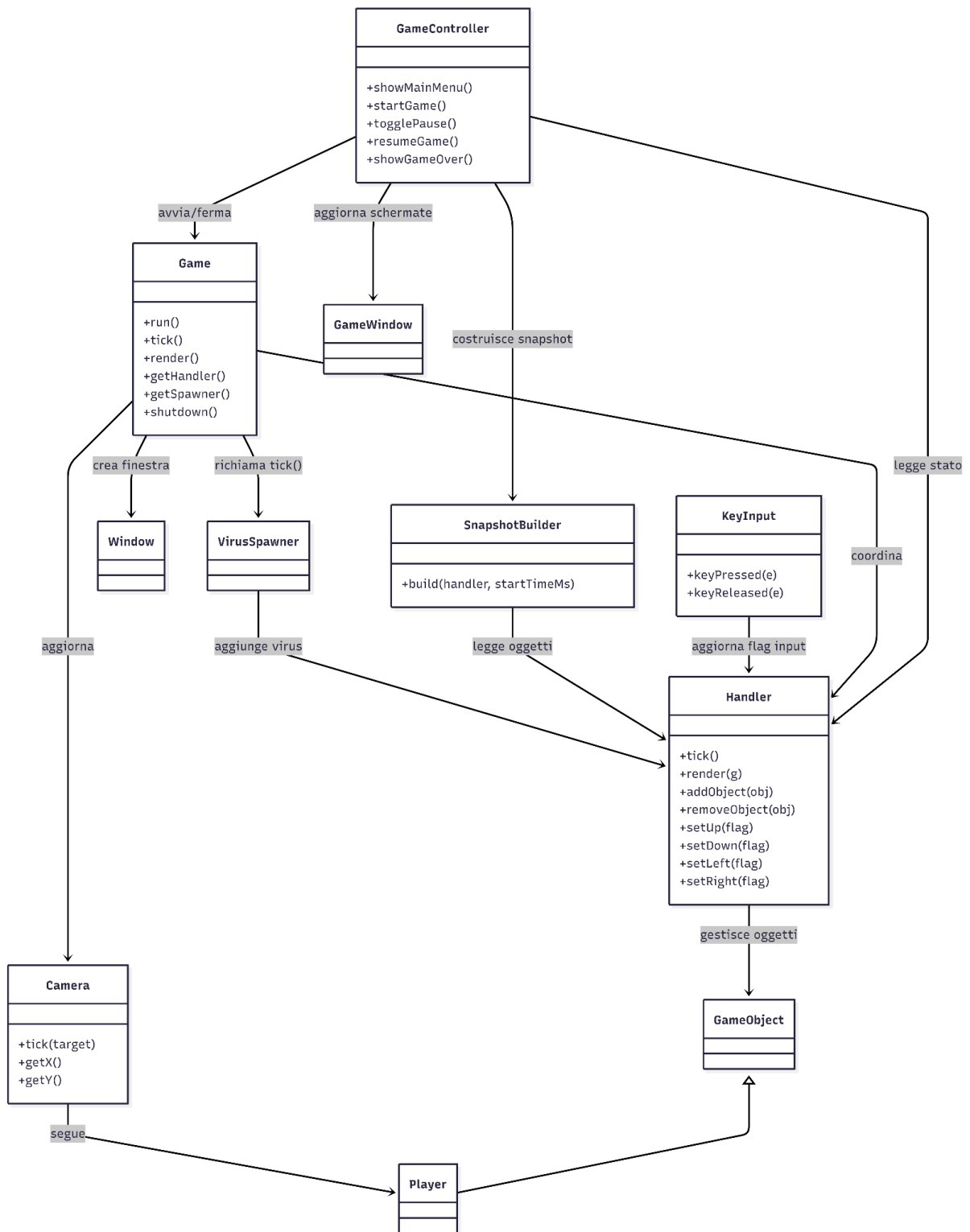
`View` è responsabile della rappresentazione grafica, `Window` inizializza la finestra di gioco mentre la classe `Camera` gestisce la porzione di mondo visibile mantenendo il player al centro della schermata e limita il suo spostamento ai bordi del livello.

La gestione degli input è demandata alla classe `KeyInput` che, aggiorna nell'`Handler` i flag corrispondenti ai tasti premuti.

Queste informazioni vengono poi lette dal player nella fase di aggiornamento, consentendo così di separare il rilevamento degli input dalla logica del movimento.

La classe `GameController` si occupa della gestione degli stati di alto livello come menu, gioco attivo, pausa e game over.

`SnapshotBuilder` costruisce una rappresentazione dello stato corrente utile per la componente grafica e all'`HUD`.



L'architettura del progetto può essere definita, quindi, come una struttura MVC-like, integrata con un game loop centrale ed un controller dedicato alla gestione degli stati applicativi.

Gerarchia delle entità di gioco e gestione del ciclo

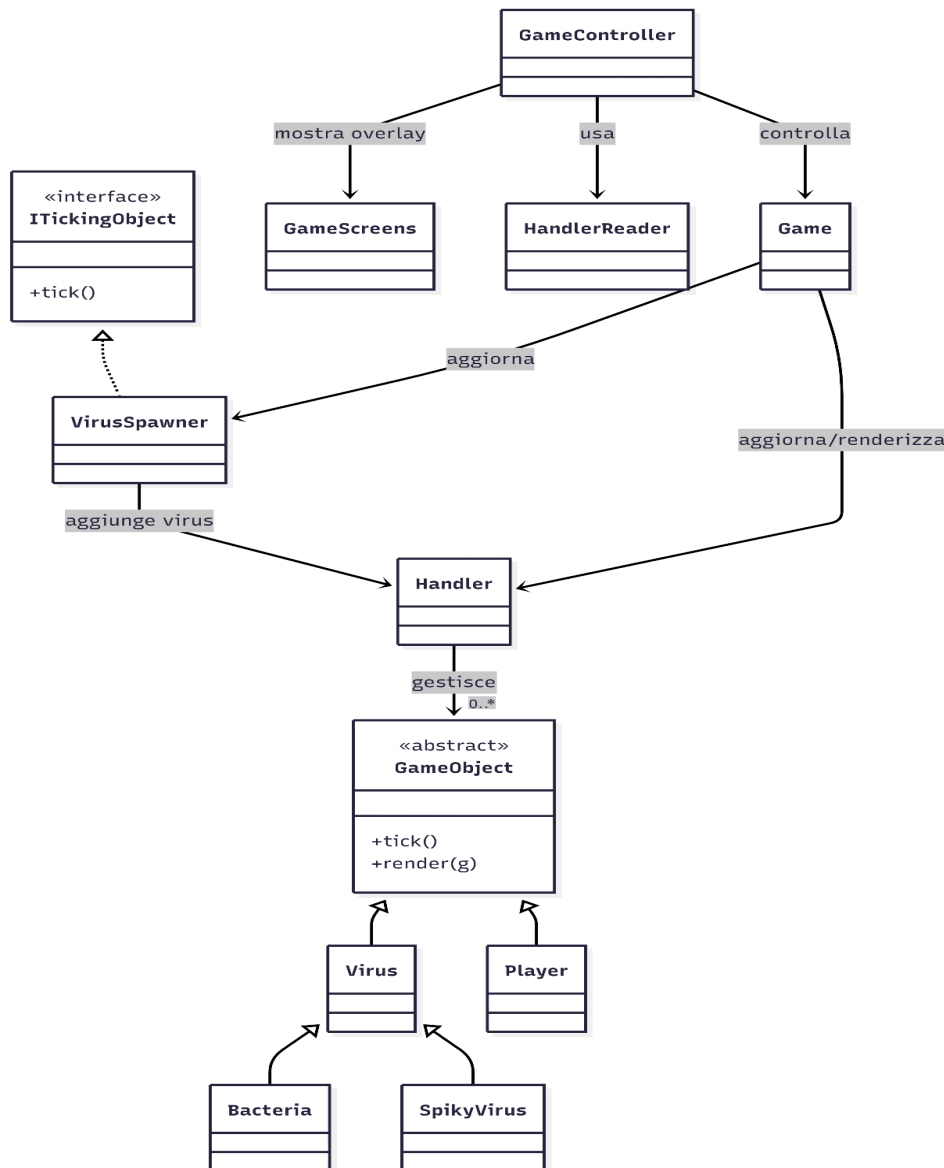
L'architettura del progetto distingue tra le entità presenti nel mondo di gioco e le componenti che governano il ciclo di vita complessivo della partita. Le entità della scena sono organizzate attorno alla classe astratta `GameObject`, che rappresenta la base comune per gli oggetti presenti nel mondo di gioco e gestiti centralmente dall'Handler. Quest'ultimo mantiene infatti la collezione degli oggetti attivi e, a ogni iterazione del game loop, richiama i metodi `tick()` e `render()` di ciascun elemento.

Per rendere più esplicita la distinzione tra comportamento logico e rappresentazione grafica, nel progetto è stata introdotta anche l'interfaccia `ITickingObject`, che identifica le componenti aggiornabili a ogni ciclo di gioco. Tale interfaccia costituisce una base concettuale utile per modellare in modo uniforme sia le entità visibili sia gli oggetti puramente logici; tuttavia, nella versione corrente del progetto, la sua integrazione è solo parziale, poiché il motore continua a basarsi principalmente sulla gerarchia `GameObject` e sulla gestione centralizzata effettuata dall'Handler.

Un caso significativo è rappresentato da `VirusSpawner`, che implementa il comportamento temporale di generazione dei nemici ma non appartiene alla gerarchia degli oggetti grafici. Per questo motivo esso viene aggiornato separatamente dalla classe `Game`, che nel proprio metodo `tick()` richiama sia `handler.tick()` per gli oggetti della scena sia `spawner.tick()` per la logica di spawn. Questa soluzione consente di mantenere separata la componente puramente logica dalla rappresentazione del mondo di gioco, pur senza rifattorizzare completamente il motore su un'unica astrazione comune.

Accanto al motore vero e proprio, la classe `GameController` gestisce invece il ciclo di vita applicativo della partita, definendo gli stati principali `MENU`, `PLAYING`, `PAUSED` e `GAME_OVER`. Essa coordina l'avvio di una nuova partita, l'estrazione dell'Handler tramite `HandlerReader`, la sospensione temporanea degli oggetti durante la pausa e la visualizzazione degli overlay grafici, implementati nella classe `GameScreens`, per menu principale, pausa, scelta degli upgrade e schermata finale.

Nel complesso, questa organizzazione mostra una distinzione già ben presente tra logica interna del motore, gestione delle entità di gioco e controllo degli stati applicativi. Le interfacce introdotte rappresentano quindi un passo verso un'architettura più uniforme, ma nella versione corrente restano soprattutto un supporto progettuale e una base per futuri refactoring, più che un meccanismo pienamente adottato in tutto il codice.



Lo schema mette in evidenza la coesistenza di una gerarchia tradizionale basata su `GameObject`, utilizzata dal motore per la gestione delle entità presenti nella scena, e di componenti logiche o di controllo, come `VirusSpawner` e `GameController`, che governano rispettivamente la generazione dei nemici e il ciclo di vita complessivo della partita.

Gestione degli stati del gioco

Per controllare il ciclo di vita dell'applicazione è stata introdotta nella classe `GameController` una macchina a stati finiti semplificata, basata sui quattro stati principali `MENU`, `PLAYING`, `PAUSED` e `GAME_OVER`. Questa struttura consente di separare chiaramente la fase di menu iniziale, la partita in corso, la sospensione temporanea del gioco e la conclusione della sessione.

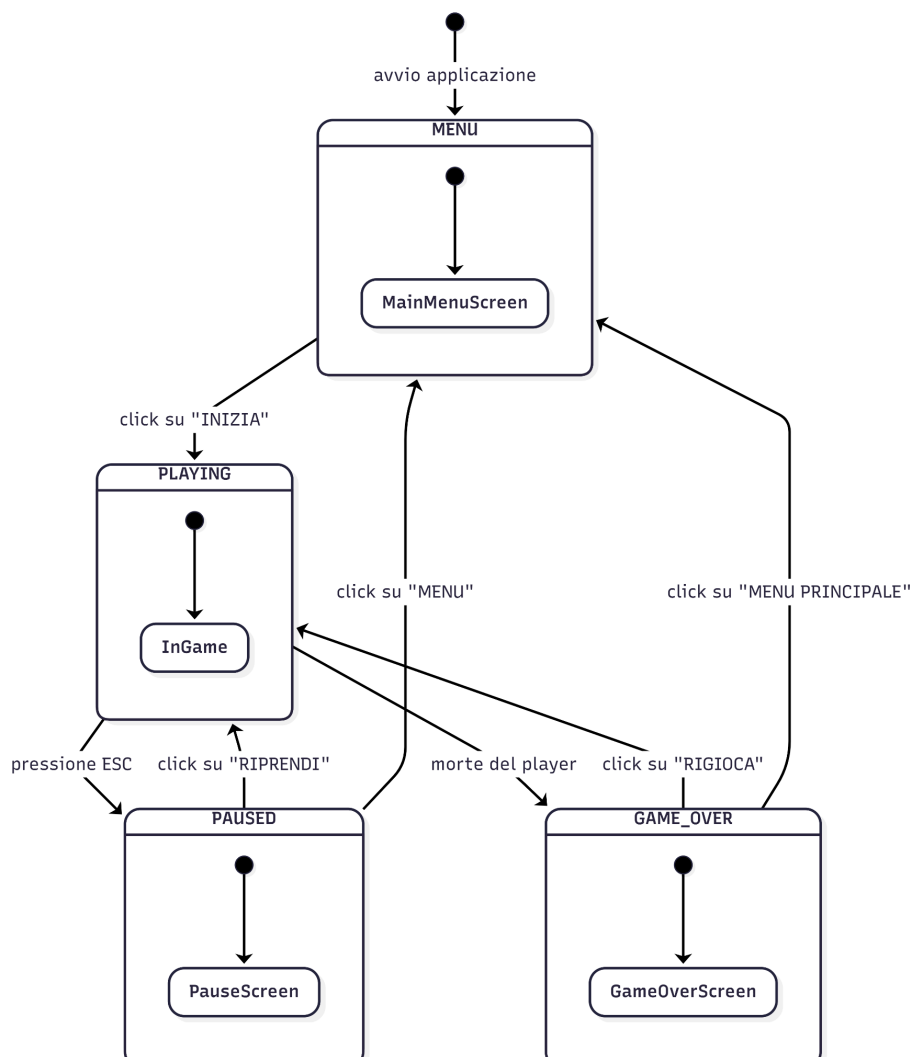
All'avvio dell'applicazione il sistema entra nello stato `MENU`, nel quale viene mostrata la schermata principale tramite l'overlay `MainMenuScreen`. Quando l'utente seleziona l'avvio

della partita, il controller crea una nuova istanza di Game, ottiene il relativo Handler e porta il sistema nello stato PLAYING, in cui il gioco viene aggiornato normalmente e l'interfaccia HUD viene refreshata periodicamente tramite un timer dedicato.

Durante lo stato PLAYING, la pressione del tasto ESC provoca il passaggio a PAUSED. In questa fase il controller salva temporaneamente gli oggetti presenti nell'Handler, ne sospende l'aggiornamento e mostra l'overlay PauseScreen, da cui il giocatore può scegliere se riprendere la partita oppure tornare al menu principale.

Lo stato GAME_OVER viene invece raggiunto quando il metodo checkGameOver() rileva che il player è morto. In questo caso il controller arresta il timer di aggiornamento dell'interfaccia e mostra l'overlay GameOverScreen, dal quale è possibile avviare una nuova partita oppure ritornare al menu iniziale.

La presenza della schermata LevelUpScreen suggerisce inoltre un'estensione possibile della macchina a stati, con l'introduzione di uno stato dedicato alla selezione dei potenziamenti. Nella versione attuale, tuttavia, tale schermata rappresenta un overlay specializzato della view e non uno stato esplicitamente modellato nell'enumerazione del GameController.



2.2 Design dettagliato

Valentina Rigato:

La mia parte di sviluppo riguardava:

- Camera e motore di gioco
- Creazione dei nemici e relativa generazione

Virus: è stata creata la classe astratta Virus (che implementa l'interfaccia IVirus) per definire i metodi comuni a tutte le classi che estendono la abstract class: SpikyVirus, Bacteria e BossVirus.

Problematica: creare tipi di mostri diversi durante tutto l'arco temporale di gioco.

Soluzione: la classe VirusSpawner genera mostri diversi con aspetti diversi in base allo scorrere del tempo. Nella prima fase vengono creati principalmente virus di tipo SpikyVirus, mentre nella seconda fase vengono introdotti anche i Bacteria; al raggiungimento di una soglia temporale prefissata viene inoltre generato un BossVirus. In questo modo la difficoltà cresce in maniera graduale e controllata.

Problematica: posizionare i mostri e fare in modo che raggiungano il player.

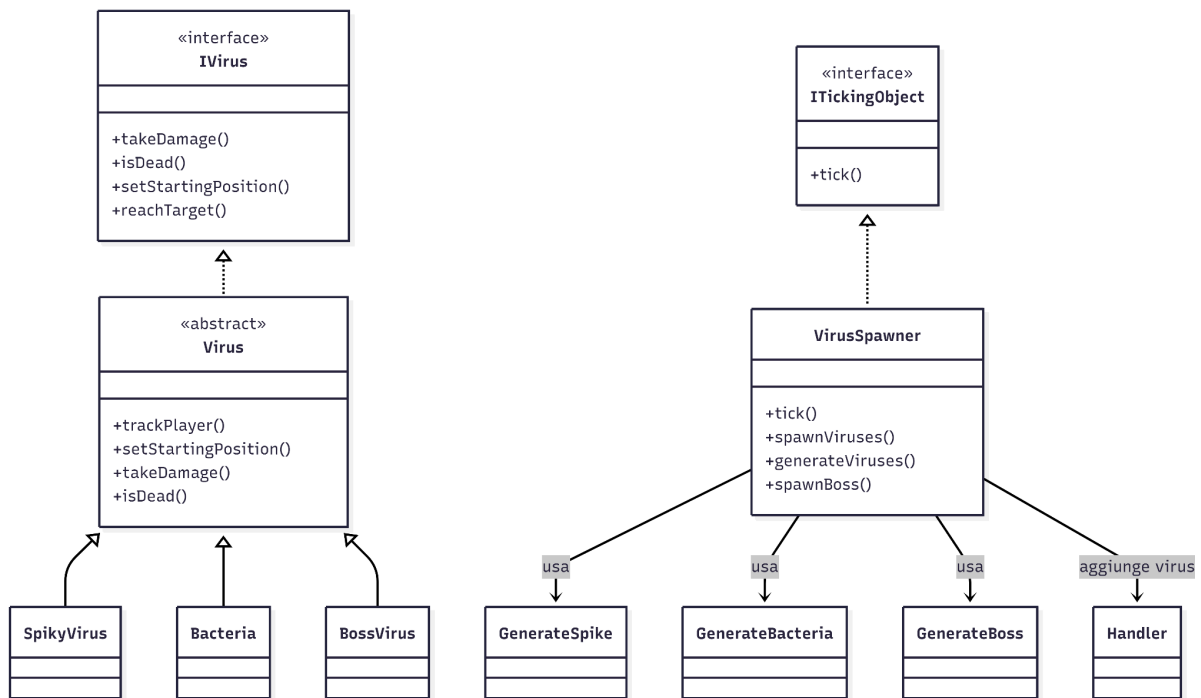
Soluzione:

la classe Virus mette a disposizione metodi comuni come setStartingPosition(), che genera una posizione casuale in un anello attorno al giocatore, e trackPlayer(), che calcola la direzione verso il player normalizzando il vettore fra nemico e bersaglio. In questo modo i virus nascono attorno al personaggio e iniziano a inseguirlo in maniera uniforme, lasciando alle sottoclassi la possibilità di introdurre eventuali variazioni nel comportamento.

Problematica: separare la logica di aggiornamento dello spawner dalla gerarchia degli oggetti grafici.

Soluzione: VirusSpawner implementa l'interfaccia ITickingObject così da rappresentare una componente che partecipa al game loop solo dal punto di vista logico.

La classe spawner non appartiene alla gerarchia di GameObject ma viene aggiornato tramite il metodo tick().



Camera: la classe Camera rappresenta la telecamera di gioco ed è responsabile di mantenere il player al centro della schermata e di gestire lo spostamento della visualizzazione di quest'ultimo in base al movimento del giocatore.

Problema: centrare la camera rispetto al giocatore.

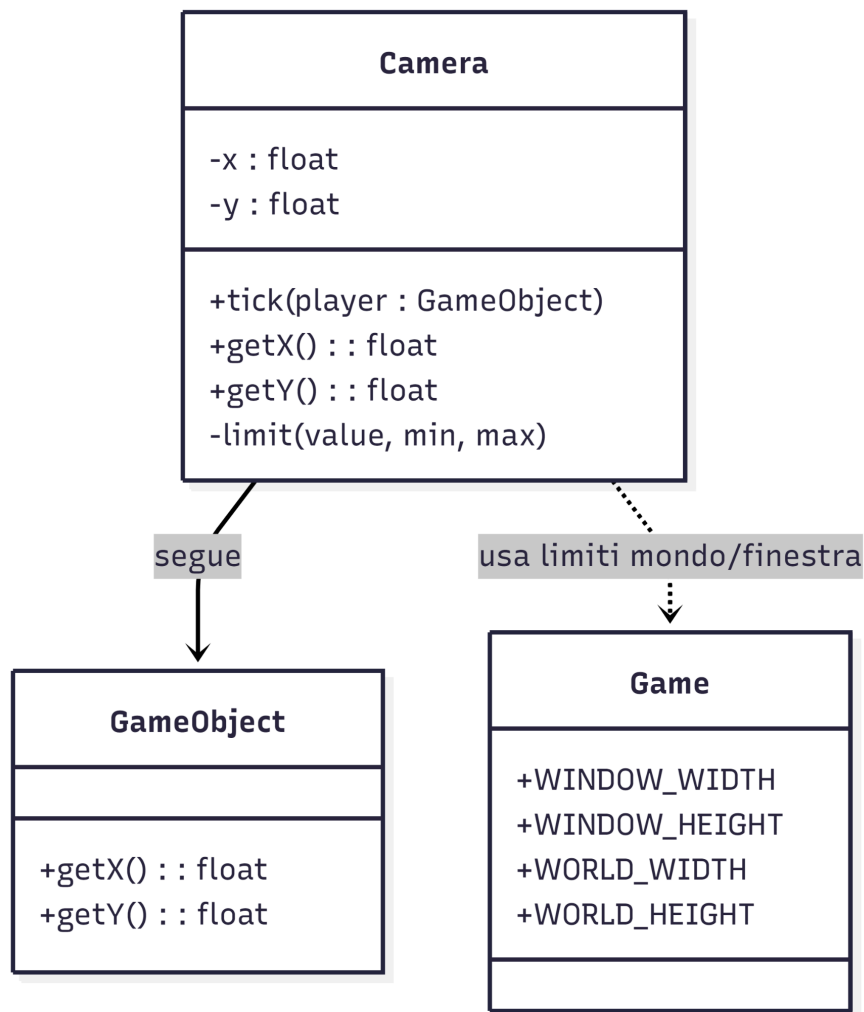
Soluzione: nel metodo tick() viene calcolata una posizione target sottraendo alle coordinate del player metà della larghezza e metà dell'altezza della finestra di gioco.

Così facendo la telecamera non cerca di posizionarsi direttamente sulle coordinate del personaggio ma su un offset in modo che il player risulti al centro.

Problematica: impedire che la visualizzazione esca dai bordi della mappa quando il player si avvicina ai bordi dell'arena.

Soluzione: dopo l'aggiornamento della posizione la camera applica un controllo di clamping attraverso il metodo limit(), vincolando i valori di x e y all'interno di un intervallo compreso tra 0 e la differenza tra dimensioni del mondo e dimensioni della finestra.

In questo modo la telecamera rimane confinata all'interno dei limiti dell'arena.



Rebecca Scarcelli:

PlayerStats

Si tratta della classe che raccoglie tutte le statistiche del personaggio. Oltre che ai getter e setter presenta tre metodi che racchiudono la logica di gioco principale:

1. `computeDamage()`: applica il danno inflitto ai nemici
2. `receiveDamage()`: applica il danno ricevuto dai nemici riducendolo in base alle difese del personaggio. Il metodo garantisce che il personaggio riceva sempre almeno 1 danno.
3. `applyLifesteal()`: rigenera i punti vita (HP) del personaggio in base al danno inflitto ai nemici
4. `isAlive()`: funzione booleana che controlla se il personaggio è vivo o meno

Jonathan Salvetti:

il GameSnapshot: la parte grafica del gioco (arena, HUD, schermate) ha bisogno a ogni frame di molte informazioni sparse nel model: posizione e vita del player, posizione, HP e tipo di ogni virus, proiettili in volo, arma equipaggiata, tempo di sopravvivenza. Un accesso diretto della View agli oggetti di gioco avrebbe creato un forte accoppiamento: ogni modifica alle classi del model avrebbe rischiato di rompere il rendering, e la View avrebbe letto strutture dati mentre il game loop le stava modificando da un altro thread.

Soluzione: ho introdotto un oggetto GameSnapshot, una "fotografia" dello stato di gioco in un dato frame. Lo snapshot viene costruito dal Controller tramite la classe SnapshotBuilder, che legge esclusivamente l'API pubblica già esposta dal model (Handler, Player, Virus, Projectile) e la traduce in semplici classi-dati pensate per il rendering (EnemyData, ProjectileData). La View (ArenaPanel, HudPanel) riceve solo lo snapshot e non conosce in alcun modo le classi del model. Il Controller fa da traduttore tra i due mondi. Come si vede nel diagramma UML, non esistono frecce dirette tra View e Model: passano tutte per lo snapshot. Un vantaggio concreto emerso durante lo sviluppo: quando è cambiato il costruttore di Projectile con l'aggiunta del nome dell'arma, la View non ha richiesto alcuna modifica, perché dipende solo da ProjectileData.

Il model non espone gli HP massimi del player, necessari per la barra della vita. Ho risolto memorizzando il primo valore di HP osservato per ogni istanza, la scelta della mappa "debole" evita che il builder trattenga in memoria riferimenti a oggetti che il gioco ha già eliminato.

Gestione degli stati di partita nel GameController

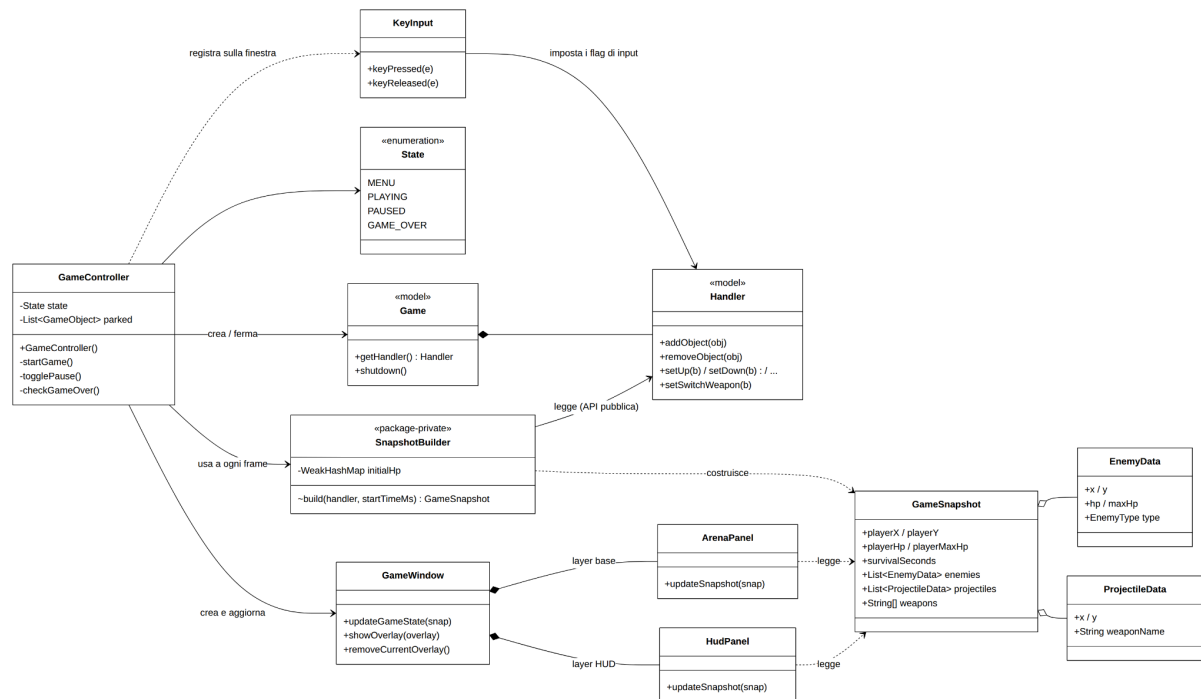
Problema: il gioco deve attraversare stati diversi (menu, partita, pausa, game over) e il motore di gioco non prevedeva né pausa né riavvio; non potevo stravolgere una classe non mia

quindi GameController modella gli stati e centralizza tutte le transizioni di avvio partita, pausa e ripresa con ESC, vittoria a tempo dopo 90 secondi, morte del player, ritorno al menu. Per la pausa ho adottato una soluzione non invasiva: gli oggetti di gioco vengono temporaneamente "parcheggiati" fuori dalla lista dell'Handler (così il game loop del model non li aggiorna) e reinseriti alla ripresa. In questo modo ho ottenuto la pausa senza modificare una riga del motore.

Arena, HUD sempre visibile (menu, pausa, level-up, game over) devono convivere nella stessa finestra sovrapponendosi senza interferire; le schermate inoltre non devono conoscere il Controller, altrimenti si crea una dipendenza tra View e Controller.

Soluzione: GameWindow usa un JLayeredPane con tre livelli: l'arena sul livello base, l'HUD su un livello superiore sempre visibile, e la schermata attiva sul livello modale, gestita con showOverlay() e removeCurrentOverlay(). Le schermate sono JPanel autonomi che ricevono il comportamento dall'esterno tramite callback (Runnable per inizia/riprendi/menu, IntConsumer per la scelta dell'upgrade): è un'applicazione dell'inversione delle dipendenze, che rende le schermate riusabili e testabili senza Controller.

KeyInput estende KeyAdapter e traduce gli eventi Swing in semplici flag booleani sull'Handler (su, giù, destra, sinistra, cambio arma), che il Player legge nel proprio tick(). Questa scelta separa il momento in cui viene premuto un tasto dal momento in cui l'input ha effetto (game loop), evitando problemi di concorrenza e mantenendo il model indipendente da Swing.



Francesco Prandini :

La mia parte di sviluppo riguardava:

- gestione degli HP e del danno del player;
- creazione delle armi e dei proiettili;
- gestione delle collisioni tra player, virus e proiettili.

Player: la classe gestisce i punti vita del giocatore, il movimento, lo sparo automatico e il cambio dell'arma. Gli HP non possono scendere sotto zero e non è possibile applicare un danno negativo.

Problematica: gestire più armi senza inserire direttamente nel Player valori fissi di danno e velocità.

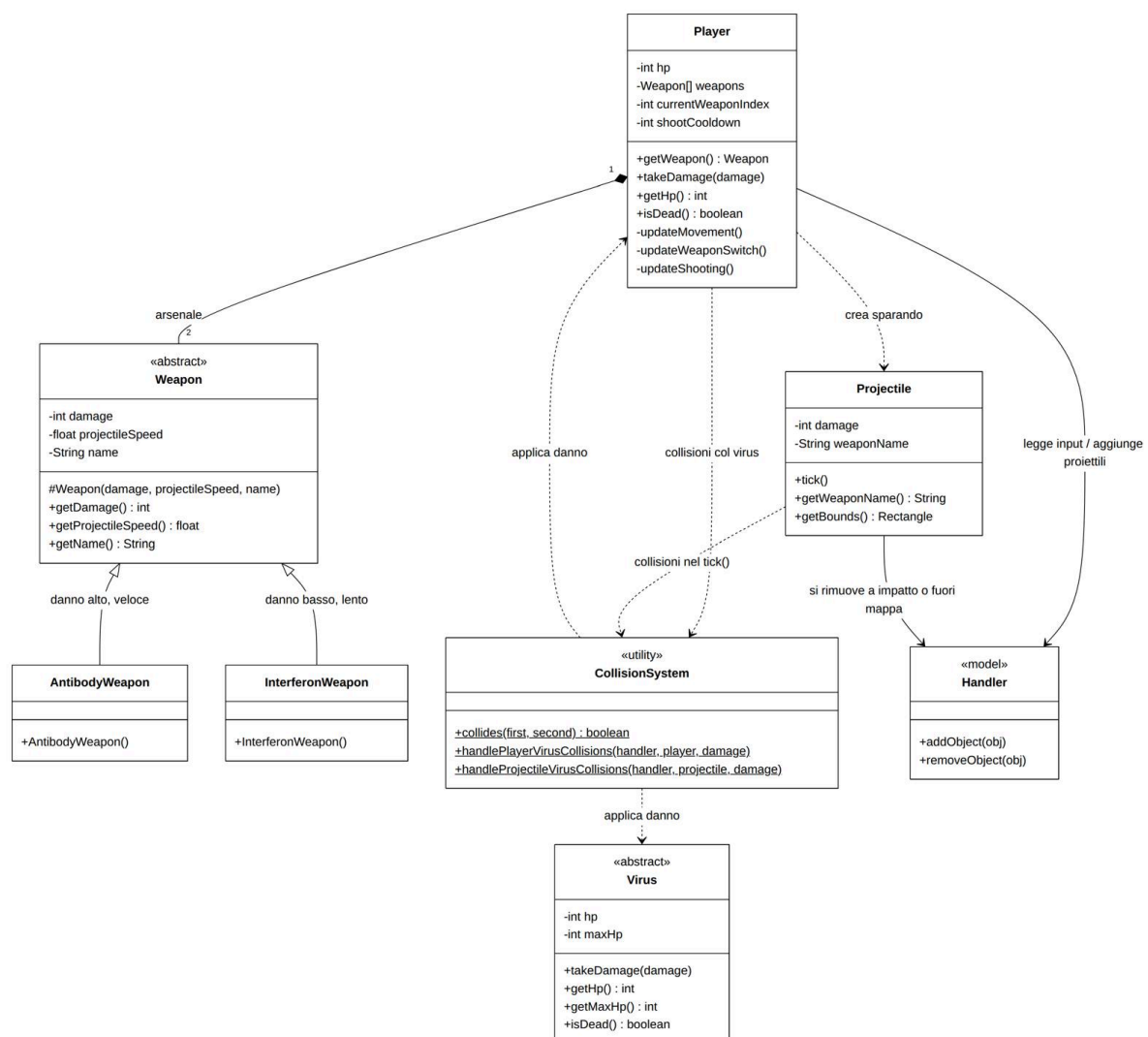
Soluzione: è stata creata la classe astratta Weapon, che contiene nome, danno e velocità del proiettile. AntibodyWeapon e InterferonWeapon rappresentano le due armi concrete. Il player conserva entrambe e può cambiare quella equipaggiata tramite il tasto Q.

Projectile: rappresenta i proiettili generati dalle armi. Ogni proiettile conserva velocità, danno e nome dell'arma che lo ha generato. Viene rimosso quando colpisce un virus oppure quando esce dai limiti del mondo.

Problematica: evitare di ripetere la logica delle collisioni dentro Player e Projectile.

Soluzione: è stata creata la classe CollisionSystem, che centralizza il controllo delle collisioni attraverso le hitbox degli oggetti. In caso di collisione tra player e virus viene applicato il danno al player, mentre un proiettile danneggia il virus e viene successivamente rimosso.

Durante lo sviluppo era stata inizialmente utilizzata una copia della lista degli oggetti per eseguire i controlli. Questa soluzione causava però un forte rallentamento con molti proiettili, quindi è stata sostituita con un ciclo indicizzato sulla lista dell'Handler.



3.1 Testing automatizzato

Valentina Rigato:

- settaggio ambiente di test Junit
- CameraTest: per poter controllare che la camera seguisse il movimento del giocatore e che venissero rispettati i bordi dell'arena
- VirusTest: i virus una volta danneggiati dovrebbero avere una riduzione della barra di vita, la barra dell'HP del boss finale dovrebbe essere più grande, e la barra della vita non dovrebbe scendere al di sotto di zero
- VirusSpawnerTest: il boss finale dovrebbe comparire dopo un certo tempo dall'avvio del gioco, lo spawn dei nemici esiste ma il boss finale viene spawnato solo una volta alla fine.

Francesco Prandini :

- PlayerTest: verifica la riduzione degli HP, il limite minimo pari a zero, il riconoscimento della morte, il rifiuto dei danni negativi e il cambio dell'arma una sola volta per ogni pressione del comando.
- WeaponTest: verifica che AntibodyWeapon e InterferonWeapon restituiscano correttamente nome, danno e velocità del proiettile.
- ProjectileTest: verifica il movimento del proiettile, il nome dell'arma che lo ha generato e il comportamento durante le collisioni con i virus.
- CollisionSystemTest: verifica che due hitbox sovrapposte siano riconosciute come in collisione e che due hitbox separate non lo siano.

Jonathan Salvetti:

- KeyInputTest: per controllare che pressione e rilascio di WASD, frecce e del tasto Q aggiornino correttamente i flag di movimento e cambio arma nell'Handler, che i tasti non mappati vengano ignorati, e che due direzioni opposte possano risultare premute insieme (la risoluzione spetta al Player);
- SnapshotBuilderTest: per controllare la traduzione model>snapshot: posizioni centrate sugli sprite, HP e arma del player, la "memoria" degli HP massimi dopo un danno, la classificazione dei nemici, i dati dei proiettili, il calcolo dei secondi di sopravvivenza e il vincolo che il builder richiede la presenza del player nell'Handler
- ViewDataTest: per controllare che le classi-dati passate alla View (EnemyData, ProjectileData, UpgradeOption) memorizzino correttamente i valori dei costruttori
- N.B: le classi grafiche (ArenaPanel, HudPanel, GameScreens, GameWindow) e il GameController non hanno test automatici perché istanziarli apre vere finestre Swing; sono stati verificati manualmente. Per questo SnapshotBuilder e KeyInput sono stati progettati senza dipendenze grafiche, in modo da essere testabili

3.2 Metodologia di lavoro

Il gruppo ha lavorato in modo coeso sia durante la scelta del tipo di progetto da realizzare che successivamente per la stesura di una lista di funzionalità minime da portare a termine. Ciascun membro ha scelto da questa lista le parti sulle quali avrebbe lavorato. Una volta delineato questo, sempre in gruppo, abbiamo deciso un design architettonico da seguire.

Lungo il corso di tutto il progetto sono stati utilizzati un gruppo Whatsapp per la comunicazione quotidiana ed un canale Discord per le riunioni settimanali in più è stato redatto un documento condiviso su Google Drive per appunti e condivisione del materiale utile. Dato che i membri del gruppo vivono lontani tra di loro e tre sono lavoratori non è stato possibile organizzare incontri dal vivo.

Per condividere il codice è stato usato fin da subito Git. Ci siamo impegnati a non lavorare direttamente sul branch main e a creare per ciascuna feature un branch separato. Di seguito è indicata nel dettaglio la suddivisione dei compiti:

Valentina Rigato:

- Impostazione della repository di progetto e supporto ai colleghi nell'utilizzo di Git
- Gestione del game engine
- Gestione della camera di gioco
- Gestione di spawn dei nemici e creazione di due tipologie di nemici

Francesco Prandini:

- Gestione degli HP e del danno del player;
- Creazione delle classi relative alle armi;
- Gestione dei proiettili e dello sparo automatico;
- Gestione delle collisioni tra player, virus e proiettili;
- Implementazione del cambio arma;

Jonathan Salvetti:

- Gestione dell'input da tastiera
- Realizzazione dell'interfaccia grafica: arena, HUD e schermate di gioco
- Gestione degli stati di partita tramite il GameController
- Creazione del GameSnapshot per passare i dati dal model alla view

3.3 Note di sviluppo

Per facilitarci nello sviluppo del gioco, soprattutto nelle fasi iniziali, abbiamo seguito dei tutorial qui di seguito la fonte che abbiamo utilizzato.

Per la creazione da zero del motore di gioco e del motore grafico abbiamo preso spunto da due tutorial presenti su Youtube:

– [Java Game Programming Wizard Course](#)

In particolare con l'aiuto di questo tutorial sono state poste le basi delle classi Game, GameObject, Handler, KeyInput usate per ottenere una versione rudimentale del motore di gioco e del suo motore grafico. Da quel momento in poi le classi sono state fortemente riviste nelle funzionalità e riadattate ad una logica OOP.

Valentina Rigato:

- Utilizzo di Random:
 - <https://github.com/valentina-rg/OOP25-bioassault/blob/6a2f8d7286bd1d1f888bc2e60e52ac0054cefb9a/src/main/java/it/unibo/bioassault/model/viruses/Virus.java#L9>
 - <https://github.com/valentina-rg/OOP25-bioassault/blob/6a2f8d7286bd1d1f888bc2e60e52ac0054cefb9a/src/main/java/it/unibo/bioassault/model/viruses/VirusSpawner.java#L6>
-

Jonathan Salvetti:

Per lo sviluppo della parte grafica e dell'input non ho seguito tutorial specifici, ma mi sono basato sulla documentazione ufficiale Oracle (The Java Tutorials), in particolare le sezioni Performing Custom Painting per il rendering dei pannelli, How to Use Layered Panes per la sovrapposizione di arena, HUD e schermate, How to Write a Key Listener per l'input da tastiera e How to Use Swing Timers per le animazioni e il loop dell'HUD.

Riferimenti e tutorial utilizzati:

- <https://docs.oracle.com/javase/tutorial/uiswing/painting/index.html>
- <https://docs.oracle.com/javase/tutorial/uiswing/components/layeredpane.html>
- <https://docs.oracle.com/javase/tutorial/uiswing/events/keylistener.html>
- <https://docs.oracle.com/javase/tutorial/uiswing/misc/timer.html>

4 Commenti finali

4.1 Autovalutazione e lavori futuri

Valentina Rigato: Sono entrata all'interno di un gruppo già formato ed ho proposto l'idea di gioco sulla quale lavorare. Durante lo svolgimento del lavoro ho organizzato gli incontri settimanali ed ho cercato di dare supporto ai miei colleghi.

Sono soddisfatta del lavoro svolto e del clima sereno e collaborativo tra di noi.

Non avevo mai sviluppato un gioco prima d'ora ed è stato appassionante ed è stata un'occasione per imparare molto.

Il mio punto debole è stata, per l'appunto, la mia totale inesperienza circa lo sviluppo di un videogame che non mi ha permesso di delineare le priorità circa lo sviluppo.

Se dovessi continuare a lavorare al progetto migliorerei alcune funzionalità circa il motore di gioco.

Francesco Prandini:

Mi sono occupato principalmente del sistema di combattimento, lavorando sulla gestione degli HP, del danno, delle armi, dei proiettili e delle collisioni.

La difficoltà tecnica principale è stata la gestione delle collisioni, perché una prima soluzione provocava un forte rallentamento quando aumentava il numero di proiettili. Dopo aver individuato il problema ho modificato il modo in cui veniva attraversata la lista degli oggetti. Ho trovato impegnativa anche l'integrazione con le parti sviluppate dagli altri membri. Non conoscendoci prima del progetto, non è stato sempre semplice coordinarci e conoscere lo stato reale delle diverse funzionalità.

Sono comunque soddisfatto della parte realizzata. Se continuassi il progetto migliorerei il bilanciamento delle armi, aggiungerei altri test e completerei il sistema di esperienza e level-up.

Jonathan Salvetti: Mi sono occupato dell'input da tastiera, dell'intera parte grafica (arena, HUD e schermate di menu, pausa, level-up e game over) e del GameController che coordina gli stati della partita.

Il mio punto debole oltre al fatto che fosse la prima volta che mi cimentavo nello sviluppo di un videogame, penso sia stato cercare di coordinarsi con il gruppo, in quanto ognuno di noi avesse necessità diverse. Come lavori futuri mi piacerebbe completare il collegamento della schermata di level-up, già pronta lato grafico, al sistema di esperienza.

Rebecca Scarcelli: Sono entrata in un gruppo già semi formato di persone che non conoscevo, nonostante tutto tutti i membri del gruppo hanno tentato di darsi supporto a vicenda quindi sono stata piacevolmente sorpresa delle modalità con cui abbiamo lavorato. Nonostante tutto ho avuto alcune difficoltà nell'organizzazione e nel rispetto delle tempistiche per le parti a me assegnate a causa di lavoro e altri esami.

È stata tuttavia una preziosa occasione per imparare e perfezionare il modo in cui scrivo il codice.

Se dovessi continuare il progetto migliorerei alcune funzionalità sull'applicazione degli upgrade e la possibilità di apportare modifiche agli upgrade predefiniti.

A Guida utente

Il giocatore deve sfuggire ai virus muovendosi con le frecce direzionali o con i comuni tasti W(su) A(sinistra) S(giù) D(destra).

Il player inizia il gioco in un punto dell'arena, all'arrivo dei nemici si troverà a dover sparare per potersi difendere. C'è un progressivo aumento della difficoltà causato dalla moltiplicazione dei mostri da sconfiggere. Il virus muore se la sua barra della vita raggiunge lo zero.

Il gioco finisce se il player esaurisce la sua vita a disposizione (il giocatore muore) o dopo un minuto e mezzo di gioco sopravvivendo all'arrivo dell'ultima ondata di nemici.

Come indicato in basso nella schermata:

- Esc: mette in pausa il gioco
- Ci sono indicazioni su come muovere il player nell'arena
- Il player può cambiare arma tramite il tasto Q